Experiment No.03

Aim: Apply Decision Tree and Random Forest for classification tasks.

Theory:

1.Data Source

Dataset: Heart Disease
Source: Kaggle

Link: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

2. Dataset Description

The Heart Disease Dataset is a supervised learning dataset used for classification problems in machine learning. The goal is to predict the presence of heart disease based on several health parameters.

```
df = pd.read_csv("sample_data/heart.csv")

df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

Target Variable

- 0 → No Heart Disease
- 1 → Presence of Heart Disease

This dataset is commonly used for binary classification tasks.

3. Mathematical Formulation of the Algorithm

Decision Tree Classifier:

A Decision Tree splits the dataset based on feature values to classify data points.

The splitting is based on impurity measures such as Gini Index or Entropy.

Gini Index

$$Gini = 1 - \sum_{i=1}^{c} p_i^2$$

Entropy

$$Entropy = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Entropy measures the uncertainty in the dataset.

The algorithm chooses splits that maximize information gain.

Random Forest Classifier:

Random Forest is an ensemble learning method that builds multiple decision trees and combines their predictions.

The final prediction is obtained using majority voting.

Mathematically:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), ..., T_n(x))$$

Random Forest reduces overfitting and variance.

4. Algorithm Limitations

Decision Tree Limitations

- Highly prone to overfitting
- Small changes in data may produce different trees
- Performs poorly with complex relationships
- Can become very deep and complex

Random Forest Limitations

- Requires higher computational resources
- Less interpretable compared to a single decision tree
- Training time increases with large datasets
- Large number of trees may increase model complexity

5. Methodology / Workflow

The experiment followed these steps:

Step 1: Dataset Collection

The heart disease dataset was downloaded from Kaggle.

Step 2: Data Loading

The dataset was loaded into Python using the Pandas library.

Step 3: Data Preprocessing

- Checked dataset structure
- Verified missing values
- Separated input features and target variable

Step 4: Train-Test Split

The dataset was divided into:

- Training Set (80%)
- Testing Set (20%)

Step 5: Model Training

Two classification algorithms were implemented:

- Decision Tree Classifier
- Random Forest Classifier

Step 6: Model Prediction

The trained models were used to predict heart disease on the test dataset.

Step 7: Model Evaluation

Model performance was evaluated using:

- Accuracy Score
- Confusion Matrix
- Classification Report

Step 8: Visualization

Graphs were created to compare the accuracy of both models.

6. Performance Analysis

Model performance was evaluated using Accuracy Score.

Accuracy Formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

```python
from sklearn.metrics import accuracy_score

# Decision Tree Accuracy
dt_accuracy = accuracy_score(y_test, y_pred_dt)

# Random Forest Accuracy
rf_accuracy = accuracy_score(y_test, y_pred_rf)

# Store results in DataFrame
accuracy_df = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "Accuracy": [dt_accuracy, rf_accuracy]
})

accuracy_df
```

	Model	Accuracy
0	Decision Tree	0.985366
1	Random Forest	0.985366

7. Hyperparameter Tuning

Hyperparameter tuning helps optimize model performance.

Decision Tree Hyperparameters:

Important parameters:

- max_depth → controls the depth of the tree
- min_samples_split → minimum samples required to split a node
- criterion → measure of split quality (gini or entropy)

Example values tested:

max_depth = [3,5,7,10]
criterion = ["gini","entropy"]

Random Forest Hyperparameters:

Important parameters:

- n_estimators → number of trees in the forest
- max_depth → depth of trees

- min_samples_split → minimum samples required for split

Example tuning values:

n_estimators = [50,100,200]
max_depth = [5,10,15]

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

#Load Dataset
df = pd.read_csv("heart.csv")
df.head()
#Check Target Variable Distribution
sns.countplot(x="target", data=df)
plt.title("Heart Disease Distribution")
plt.show()
#Define Features and Target
X = df.drop("target", axis=1)
y = df["target"]
X.head()
#Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
X_train.head()
#Decision Tree Classifier
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test, y_pred_dt)
dt_accuracy
```

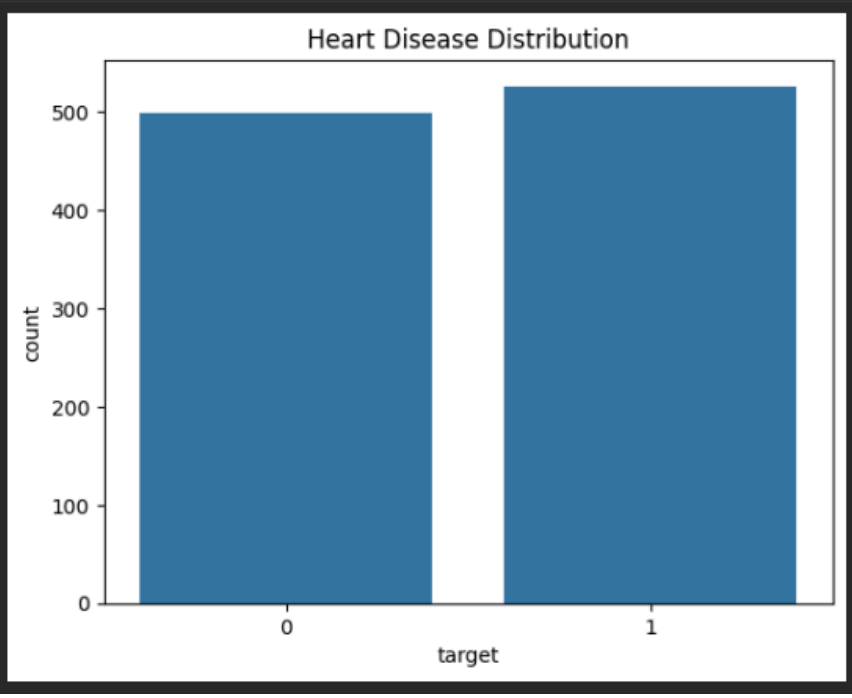
```
#Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
rf_accuracy = accuracy_score(y_test, y_pred_rf)
Rf_accuracy
#Confusion Matrix
#Decision Tree
cm_dt = confusion_matrix(y_test, y_pred_dt)
sns.heatmap(cm_dt, annot=True, fmt="d", cmap="Blues")
plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

#Random Forest
cm_rf = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm_rf, annot=True, fmt="d", cmap="Greens")
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1025 non-null   int64
 1   sex       1025 non-null   int64
 2   cp        1025 non-null   int64
 3   trestbps  1025 non-null   int64
 4   chol      1025 non-null   int64
 5   fbs       1025 non-null   int64
 6   restecg   1025 non-null   int64
 7   thalach   1025 non-null   int64
 8   exang     1025 non-null   int64
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64
 11  ca        1025 non-null   int64
 12  thal      1025 non-null   int64
 13  target    1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

```python
sns.countplot(x="target", data=df)
plt.title("Heart Disease Distribution")
plt.show()
```



```
X = df.drop("target", axis=1)
y = df["target"]

X.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2

Next steps: Generate code with X New interactive sheet

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
835	49	1	2	118	149	0	0	126	0	0.8	2	3	2
137	64	0	0	180	325	0	1	154	1	0.0	2	0	2
534	54	0	2	108	267	0	0	167	0	0.0	2	0	2
495	59	1	0	135	234	0	1	161	0	0.5	1	0	3
244	51	1	2	125	245	1	0	166	0	2.4	1	0	2

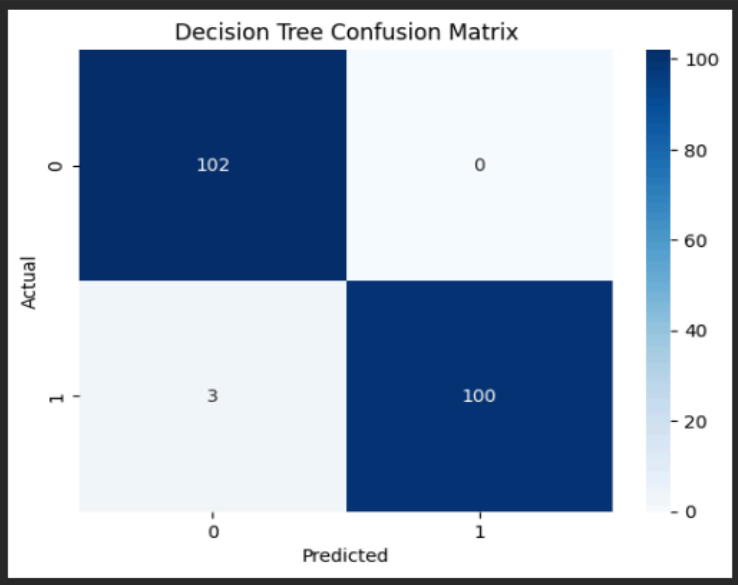
```
dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, y_pred_dt)

dt_accuracy
```

```
0.9853658536585366
```

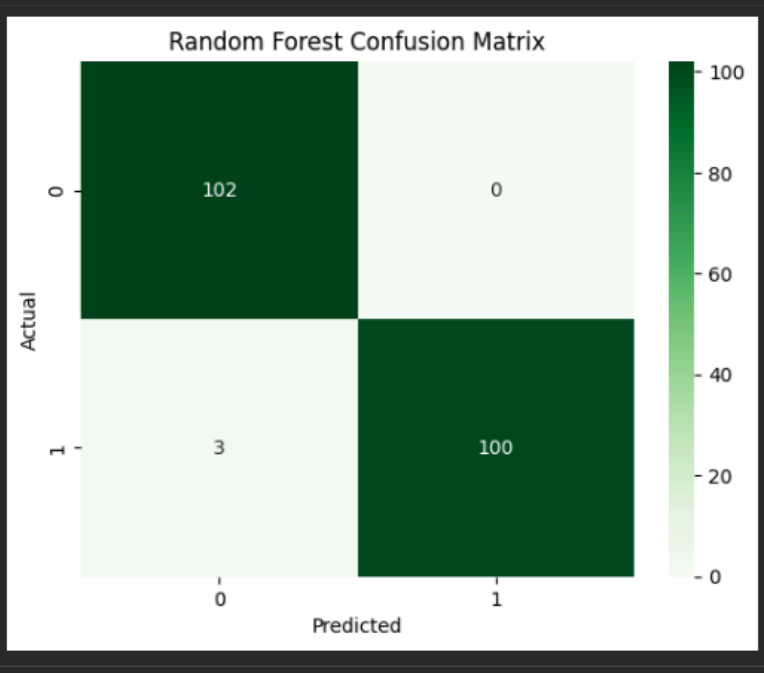
```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, y_pred_rf)

rf_accuracy
```

```
0.9853658536585366
```

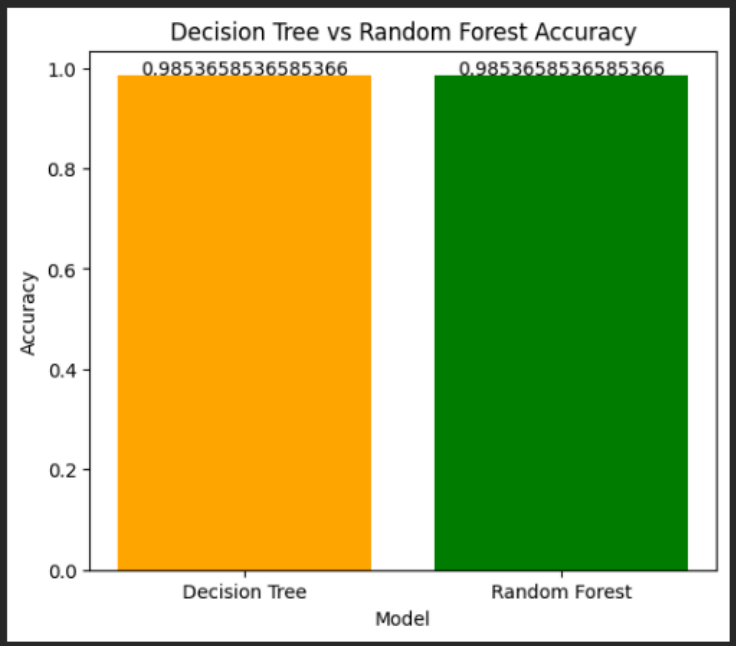
```python
cm_dt = confusion_matrix(y_test, y_pred_dt)

sns.heatmap(cm_dt, annot=True, fmt="d", cmap="Blues")
plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```python
cm_rf = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm_rf, annot=True, fmt="d", cmap="Greens")
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```python
plt.figure(figsize=(6,5))

plt.bar(
    accuracy_df["Model"],
    accuracy_df["Accuracy"],
    color=["orange", "green"]
)

plt.title("Decision Tree vs Random Forest Accuracy")
plt.xlabel("Model")
plt.ylabel("Accuracy")

for i, v in enumerate(accuracy_df["Accuracy"]):
    plt.text(i, v + 0.002, v, ha='center')

plt.show()
```



Conclusion:

In this experiment, Decision Tree and Random Forest classification algorithms were implemented to predict heart disease using medical attributes. Both models achieved high classification accuracy. Decision Trees provide easy interpretability, while Random Forest improves prediction stability through ensemble learning. The results demonstrate that ensemble methods such as Random Forest can enhance model reliability in classification problems.